Cognitive Kernel Networks (CKN)

Root Trust for Privilege-Separated Reasoning in High-Dimensional Models

John P. Alioto

Protocol v1.1.0
December 2025

Abstract

Modern neural models compute as deterministic dynamical systems in extremely high-dimensional latent spaces. Current safety and control approaches attempt to shape model behavior through natural-language instructions placed in the context window. After embedding, these instructions occupy the same representational subspace as user input and adversarial perturbations:

$$\text{safety tokens},\ \text{system prompts},\ \text{user tokens}\ \in\ \text{span}(W_E).$$

Because all conditioning signals share the same geometric substrate, they compete for influence over the hidden-state trajectory. As sequences grow, the influence of builder-provided constraints diminishes and the model's dynamics revert toward prior-dominated regions. This limitation is architectural: user-space tokens cannot enforce privilege boundaries.

This paper introduces *Cognitive Kernel Networks* (CKN), a geometric primitive for constructing privilege-separated reasoning inside high-dimensional models. Model builders control the latent dimensionality, embedding matrix, and attention geometry that define the transition function $h_{t+1} = F_\Theta(h_t, x_t)$. By choosing $\text{rank}(W_E) < \dim(\mathcal{H})$, builders can create directions in latent space that are algebraically unreachable by user input. These directions support a privileged reasoning manifold R, a mediated interface layer D, and a set of architectural invariants I. User tokens cannot perturb these regions not probabilistically, but *by construction*.

CKN does not prescribe a specific architecture, modify model parameters at inference time, or endow new reasoning capability. It provides a general-purpose geometric framework from which model builders can construct kernel–user separation analogous to operating-system privilege rings. CKN complements Cognitive Tensor Networks (CTN), which stabilize user-space trajectories through structured prefix geometry. Together, CTN and CKN form a unified perspective: both user-level and architecture-level stability are geometric control problems.

1 Introduction

Large neural models compute by evolving a hidden state through a learned vector field:

$$h_{t+1} = F_\Theta(h_t, x_t), \tag{1}$$

where h_t denotes the hidden state, x_t the input embedding, and Θ the fixed model parameters. This dynamical perspective highlights two distinct control surfaces:

1. **Initial-condition control**, which shapes the early trajectory of h_t.
2. **Topological control**, which shapes the allowable region of state-space evolution.

Cognitive Tensor Networks (CTN) address the first surface. By introducing structured prefix geometry, CTN restricts the initial hidden-state manifold and reduces drift. However, CTN

also exposed a structural limitation in modern architectures: all input tokens—system prompts, user queries, safety instructions, and adversarial prefixes—are embedded into the same manifold. There is no privileged subspace for reasoning. As a result:

- user input can perturb internal reasoning directions,
- the model lacks a "kernel mode" analogue,[1]
- prefix perturbations can deform the underlying reasoning geometry,
- and the system has no architectural root of trust.

This conflation has no analogue in secure computing. In traditional systems, user-mode applications cannot directly modify kernel memory; information flow is mediated through an interface (syscalls) with well-defined semantics. Modern neural networks lack an equivalent privilege boundary.

Cognitive Kernel Networks (CKN) address this gap. CKN introduces a mathematical primitive that defines:

- a privileged reasoning manifold R,
- a mediated interface or "driver layer" D,
- architectural invariants that constrain R,
- and a bounded-perturbation envelope guaranteeing that external tokens cannot deform R.

CKN provides the *geometry* necessary for privilege-separated reasoning. It is not a training method, not a fine-tuning recipe, and not an architectural hack. CKN is a **specification** that identifies the topological conditions a model must satisfy to support stable, predictable, and builder-governed reasoning.

In this paper we formalize CKN, establish its mathematical properties, define its axioms, and show how CTN and CKN together constitute a full-stack cognitive architecture: CTN controls the prefix geometry, and CKN establishes the kernel geometry.

The remainder of this paper proceeds as follows:

- Section 2 motivates the geometric approach to privilege separation.
- Section 3 establishes builder control and root trust.
- Section 4 derives privilege separation from root trust.
- Section 5 formalizes the privilege-separation problem.
- Section 6 introduces the CKN model and its components.
- Section 7 defines the CKN axioms, including bounded perturbation.
- Section 8 develops the nullspace and column-span results supporting internal-only kernels.
- Section 9 shows the link between CKN and Lipschitz robustness.
- Section 10 presents CKN as an architectural specification.
- Section 11 discusses implications for cognitive systems design.
- Section 12 concludes.

[1] We use "kernel" in the geometric sense of a privileged computational subspace, analogous to—but distinct from—the operating-system sense. The analogy to OS kernel space is structural rather than literal.

2 Motivation: Why Geometry Must Be Solved Geometrically

Most current safety and control approaches in machine learning rely on natural-language constraints embedded into the context window. System prompts, policy instructions, user queries, and even adversarial input all enter the model as tokens. After embedding, all such tokens occupy the same representational subspace:

$$\text{system tokens} \in \text{span}(W_E), \quad \text{safety tokens} \in \text{span}(W_E), \quad \text{user tokens} \in \text{span}(W_E).$$

Because they share the same geometric substrate, these constraints compete directly with one another. As sequence length grows, the influence of builder-provided instructions diminishes, and the trajectory of the hidden state becomes increasingly dominated by the model's learned priors.

This is an architectural limitation, not a failure of alignment technique. No amount of additional instruction-layer prompting can guarantee control within a space where all tokens share equal geometric standing.

The situation mirrors early computing systems before privilege separation. If both user processes and kernel services execute in the same address space with identical permissions, no amount of software-level policy can ensure isolation or integrity. The solution was architectural: hardware enforced privilege rings and mediated interfaces.

Cognitive Kernel Networks (CKN) propose an analogous geometric intervention for learned systems. Rather than adding more instructions into the same user-space geometry, CKN relocates privileged reasoning into a region of the latent space that user tokens cannot reach. The boundary is not a matter of policy, but of construction.

3 Builder Control and the Construction of Root Trust

CKN relies on a simple but often unstated fact: model builders have complete control over the parameters Θ that define the transition function

$$h_{t+1} = F_\Theta(h_t, x_t),$$

and therefore control the geometry of the latent space in which this computation unfolds.

The builder determines:

- the dimensionality n of the hidden state space $\mathcal{H} \subset \mathbb{R}^n$,
- the embedding matrix W_E mapping tokens to latent vectors,
- the unembedding matrix W_U mapping latent vectors to token logits,
- the structure and initialization of all attention and feedforward weights.

Users do not modify Θ. They interact only through tokenized input x_t.

Dimensionality as a Design Variable

The builder is free to choose n. Nothing requires the latent dimensionality to equal the rank of the embedding matrix. If

$$\text{rank}(W_E) < n,$$

then $\mathcal{H}$ contains directions that no external token can express.

Internal Symbols and Unreachable Directions

Let $U = \text{span}(W_E)$ denote the user-accessible subspace. If $r \in \mathcal{H}$ satisfies $r \notin U$, then no linear combination of input embeddings can produce a perturbation with any component along r. External input is algebraically confined to U.

The builder may define internal symbols—conceptual directions used during reasoning—that have nonzero components outside U. These symbols can influence computation inside the model, but users cannot express or perturb them.

Theorem (Algebraic Unreachability)

Let $\mathcal{H}$ be the latent space of dimension n, let $U = \text{span}(W_E)$ be the subspace reachable by user tokens, and let $r \in \mathcal{H}$ satisfy $r \perp U$. Then no finite sequence of user tokens can produce any hidden-state perturbation with a nonzero projection onto r.

Proof sketch. Every user input is a vector in U. Hidden states evolve under F_Θ from initial conditions that depend on these inputs, but the instantaneous perturbation contributed by external tokens remains confined to U. Since r lies strictly outside this subspace, its projection onto any user-driven perturbation is identically zero. □

Root Trust

A direction r that satisfies $r \notin \text{span}(W_E)$ is not merely hard for the user to reach—it is *unreachable by construction.* This property provides the foundation for privilege separation:

- The builder can designate regions of $\mathcal{H}$ as privileged.
- These regions cannot be moved or perturbed by user tokens.
- Computation in these regions can therefore serve as an architectural root of trust.

This root trust is purely geometric. It does not depend on alignment, policy compliance, or probabilistic behavior. It is a direct consequence of linear algebra: users cannot perturb directions that are absent from their embedding basis.

In subsequent sections we formalize how this primitive supports the construction of a privileged reasoning manifold R, a mediated interface layer D, and stable architectural invariants I.

4 From Root Trust to Privilege Separation

With root trust established, privilege separation follows from geometric construction rather than behavioral policy. Because user input is algebraically restricted to the subspace $U = \text{span}(W_E)$ while privileged directions lie outside U, the builder may define a hierarchy of reasoning spaces that the user cannot directly perturb.

Kernel Space and User Space

We define:

$$\textbf{User Space: } U = \text{span}(W_E), \qquad \textbf{Kernel Space: } R \subset \mathcal{H} \text{ with } R \not\subseteq U.$$

User tokens induce perturbations within U. Internal computation may evolve within R. The key property follows from algebraic unreachability:

$$\Pi_R(\Delta h_{\text{user}}) = 0 \quad \text{for all user-driven perturbations.}$$

Thus kernel-space computation cannot be perturbed by external input, not because of a rule or guardrail, but because the geometry of $\mathcal{H}$ forbids it.

Privilege Boundary

The privilege boundary is the set-theoretic and geometric separation between U and R:

$$U \cap R = \{0\}, \qquad R \not\subseteq U.$$

This boundary is not statistical. It is not enforced by loss penalties, behavioral policies, or alignment procedures. It arises directly from the builder's choice of embedding geometry and latent dimensionality. The user has no mechanism for generating components in R because the vocabulary does not include the basis directions needed to reach it.

Mediation and the Driver Layer

Although privileged computation occurs in R, it must interact with user-facing computation in U. We therefore define a mediated interface layer:

$$D : U \to R, \qquad D^\dagger : R \to U,$$

where D is builder-defined. The driver layer determines:

- which aspects of user input may influence privileged computation,
- which results from R may be projected back to user space,
- the allowable transformations between spaces,
- how much permeability exists across the boundary.

If D is low-bandwidth or highly filtered, user influence over kernel computation is sharply limited. If D is expressive, more nuanced interactions are possible. In all cases, control remains with the builder.

Constructing Privilege Rings

Once root trust exists, further structure is straightforward. The builder may define additional subspaces:

$$R_0 \subset R_1 \subset \cdots \subset \mathcal{H},$$

each with distinct invariants and access semantics. These rings may serve as:

- safety-critical reasoning layers,
- long-term memory or planning layers,
- model-level policy interpretation layers,
- architecturally privileged decision substrates.

Each ring inherits algebraic unreachability from the root trust primitive. The builder determines how rings communicate, which invariants they maintain, and what information they admit.

CKN as the Minimal Primitive

CKN does not prescribe:

- how R is used,
- how drivers D are implemented,
- how invariants I are chosen,
- or which privilege architecture is optimal.

CKN provides the minimal geometric primitive required for privilege separation:

$$\textbf{Unreachable directions} \quad \Rightarrow \quad \textbf{privileged subspaces} \quad \Rightarrow \quad \textbf{mediated access}.$$

This mirrors the evolution of operating systems: once the kernel/user boundary existed, architectures diverged dramatically, but all shared the same foundational idea.

In the remainder of the paper, we formalize CKN as a tuple

$$\mathbb{K} = (R, D, I, \Lambda, d),$$

encoding privileged subspaces, mediated interfaces, architectural invariants, dominance parameters, and bounded-perturbation requirements.

5 Background and Problem Statement

Modern neural architectures, including transformers and mixture-of-experts systems, operate entirely within a high-dimensional latent space $\mathcal{H} \subset \mathbb{R}^n$. All computation—attention, feedforward transformation, residual mixing, and decoding—occurs within this space. The latent dynamics are deterministic given Θ, and the system evolves a hidden-state trajectory $\{h_t\}_{t=0}^{T}$ as a function of input embeddings.

Despite their power, such architectures possess a structural deficiency that is rarely addressed explicitly: **they lack privilege separation**. User-level tokens and internal reasoning representations occupy the same manifold. This section formalizes that deficiency and its consequences.

5.1 Unified Manifold Problem

Let $\mathcal{E} : \mathcal{V} \to \mathcal{H}$ denote the embedding map from tokens to latent vectors, and let $\mathcal{U}$ denote the span of all input embeddings:

$$U = \text{span}\{\mathcal{E}(v) : v \in \mathcal{V}\}. \tag{2}$$

Similarly, let $R \subseteq \mathcal{H}$ denote the subset of latent directions used by the model for internal reasoning.

In existing architectures,

$$U \approx R, \tag{3}$$

meaning:

- user inputs are embedded into directions that directly participate in reasoning;
- internal reasoning directions can be perturbed by external tokens;
- the model has no concept of "kernel-mode" computation;
- adversarial or accidental prefixes can redirect internal geometry.

This creates a single, undifferentiated attack surface. Any sufficiently long or carefully constructed prefix can induce shifts in the latent trajectory of h_t, thereby altering the model's reasoning behavior. This phenomenon appears externally as jailbreaks, drift-collapse, or inconsistent internal coherence.

5.2 Prefix-Induced Geometry Deformation

Because U and R are entangled, user-space perturbations $\Delta x \in U$ can produce arbitrary perturbations in reasoning space:

$$\Delta h_R = \Pi_R(F_\Theta(h_t, x_t + \Delta x) - F_\Theta(h_t, x_t)), \tag{4}$$

where Π_R denotes the projection onto R. Current architectures impose no constraint ensuring that $\|\Delta h_R\|$ is bounded or aligned with the stable directions of the reasoning manifold. Thus:

1. Small perturbations in token space can produce large perturbations in reasoning space.
2. User-provided tokens can access regions of $\mathcal{H}$ that should be internal-only.
3. The model provides no architectural guarantee of stability under prefix perturbation.

This is the latent-space analogue of allowing user-mode processes to write directly into kernel memory. Such a design is unacceptable in classical computing systems and is equally problematic in cognitive systems.

5.3 Decoder-Induced Drift (Projection Collapse)

A related but distinct failure mode arises from the projection operator used to convert hidden states to output tokens. Let $\mathcal{D} : \mathcal{H} \to \mathcal{V}$ be the decoder. The projection:

$$\hat{v}_t = \mathcal{D}(h_t) \tag{5}$$

is inherently lossy. It collapses the high-dimensional structure of h_t into a discrete symbol. This collapse does *not* interfere with the geometry of $\mathcal{H}$ itself, but it *does* influence the autoregressive trajectory via:

$$x_{t+1} = \mathcal{E}(\hat{v}_t), \tag{6}$$

injecting a low-dimensional, quantized perturbation back into the state.

When the input weakly constrains the manifold, the autoregressive update dominates and can push h_t into low-density or unstable regions, resulting in externally visible errors often labeled as "hallucination." These are not psychological failures; they are projection-induced geometric failures.

Remark 1. *Hallucination is a misalignment between the projection $\mathcal{D}(h_t)$ and the stable manifold of the underlying geometry. It is a* decoder collapse*, not an indicator of model intent.*

5.4 Absence of Root Trust

The core problem is that current architectures lack a *root of trust*. There is no boundary ensuring that:

- user-space cannot perturb reasoning topology,
- internal reasoning cannot be perturbed beyond controlled bounds,
- stable computation occurs in privileged subspaces,
- the system maintains invariant structure regardless of input.

Thus the model has no analogue of:

- kernel-mode execution,
- capability rings,
- hypervisor boundaries,
- trusted computing base,
- read-only invariant regions.

This deficiency is architectural, not behavioral. No amount of alignment, RLHF, classifier gating, or prompt heuristics can fix a missing geometric boundary. Privilege separation must originate in the architecture itself.

CKN provides the mathematical primitive necessary to define such a boundary.

6 The Cognitive Kernel Network Model

Cognitive Kernel Networks provide a mathematical structure for introducing privilege separation into high-dimensional reasoning systems. CKN does not prescribe any architecture, training procedure, or enforcement mechanism. Instead, it defines the *geometric specification* that a model must satisfy in order to possess a privileged reasoning manifold insulated from user-controlled perturbations.

A CKN kernel is defined by a tuple:

$$\mathbb{K} = (R, D, I, \Lambda, d), \tag{7}$$

where:

- R is the privileged reasoning manifold,
- D is the mediated interface or "driver layer",
- I is a set of architectural invariants,
- Λ specifies gain and dominance parameters,
- d is the bounded-perturbation radius.

Each component is defined more precisely below.

6.1 Privileged Reasoning Manifold R

Definition 1 (Privileged Manifold)**.** *The* privileged reasoning manifold $R \subset \mathcal{H}$ *is a high-dimensional subspace or submanifold of the model's latent space in which internal reasoning trajectories evolve. The topology and invariant structure of* R *are determined by the model weights* Θ *and are not subject to modification by external input.*

R functions analogously to *kernel memory* in operating systems or to the *trusted computing base* in secure systems. It is the region of the latent space whose geometry must remain stable, predictable, and invariant under user-controlled perturbations.

6.2 Driver / Interface Layer D

The privileged manifold R cannot be exposed directly to the user. Instead, all access occurs through a mediated interface layer:

Definition 2 (Driver Layer)**.** *The* driver layer D *is an operator*

$$D : U \to R, \tag{8}$$

which maps user-span perturbations into a controlled subspace that R can read without allowing those perturbations to alter the topology or invariants of R. The adjoint operator

$$D^\dagger : R \to U \tag{9}$$

provides the projection of privileged computation back into the user-facing token space.

The D-layer serves as the only conduit of information between user input and privileged reasoning:

$$U \xrightarrow{D} R \xrightarrow{D^\dagger} U.$$

It is not required that D or $D^\dagger$ be linear, invertible, or representationally complete. In fact, non-invertibility is *desirable*: internal-only kernels may exist in R whose basis vectors cannot be expressed in U.

Remark 2. *The D-layer is conceptually analogous to a system-call interface: it mediates access without exposing or allowing mutation of privileged structures.*

6.3 Architectural Invariants I

The privileged manifold R possesses invariant directions and structures that must not be perturbed by user-space perturbations.

Definition 3 (Invariants)**.** *I is a set of invariant functions or vector fields on R,*

$$I = \{f_1, f_2, \ldots, f_k\}, \tag{10}$$

such that the topology, curvature, or stable directions of R are preserved under all permitted perturbations from U.

Examples include:

- fixed attractor basins,
- frozen linear subspaces,
- curvature-preserving constraints,
- invariant cones of reasoning directions.

I is purely conceptual and does not specify how invariants are enforced.

6.4 Dominance Parameters Λ

The kernel must ensure that perturbations to R remain strongly dominated by the model's internal dynamics rather than by user input.

Definition 4 (Dominance Parameters)**.** Λ *is a set of gain parameters controlling the degree to which privileged computation in R dominates user-span perturbations projected via D.*

These parameters ensure that:

$$\|\Delta h_R\| \text{ is governed primarily by } F_\Theta, \text{ not by input tokens.}$$

6.5 Bounded-Perturbation Radius d

The final component of the kernel is the perturbation bound.

Definition 5 (Bounded Perturbation)**.** *Let h_t be the privileged hidden state at time t. The kernel imposes a bound*

$$\|\Pi_R(h_{t+1} - h_t)\| \leq d, \tag{11}$$

where d is a builder-defined radius limiting how far user-induced perturbations may move the privileged state within R.

This ensures that no conceivable prefix or sequence of user-controlled inputs can "reach under" the privileged manifold.

6.6 CKN Kernel Definition

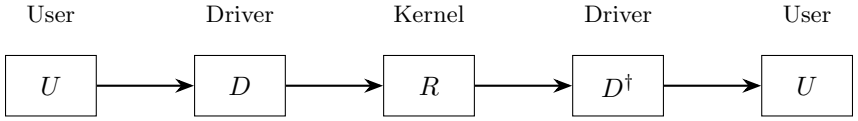
Putting the components together:

Definition 6 (CKN Kernel)**.** *A Cognitive Kernel Network is the tuple*

$$\mathbb{K} = (R, D, I, \Lambda, d)$$

that defines the geometric conditions required for privilege-separated reasoning within a high-dimensional model. The kernel defines the allowable trajectory space but does not prescribe how the model enforces these constraints.

Remark 3. *CKN is the mathematical root of trust. The weights define the privileged manifold; CKN defines the conditions under which that manifold is protected. Unless an adversary can modify Θ through physical access, they cannot "get underneath" the privileged geometry.*



$$\mathcal{H} = U \oplus D \oplus R \oplus S$$

Figure 1: CKN information flow and latent space decomposition. User input enters through U, passes through the driver layer D to reach privileged computation in R, and returns via $D^\dagger$. The full latent space $\mathcal{H}$ decomposes into user-accessible (U), driver (D), kernel (R), and auxiliary (S) subspaces.

7 Axioms of Privilege-Separated Reasoning

CKN provides a geometric specification for reasoning under privilege constraints. This section introduces the axioms governing the interaction between user-span inputs, the driver layer, and the privileged reasoning manifold.

The axioms do not prescribe implementation; they define the mathematical conditions that any CKN-compliant cognitive architecture must satisfy.

7.1 Axiom 1: Read Access (Permitted Influence)

The privileged manifold must read user-space queries; otherwise reasoning becomes decoupled from input and the system ceases to be responsive.

Axiom 1 (Read Access)**.** *For all t,*

$$\frac{\partial h_{R,t+1}}{\partial h_{U,t}} \neq 0. \tag{12}$$

User-span representations provide information to the privileged manifold through the driver interface D.

This ensures that R can interpret input while preserving its structural autonomy.

7.2 Axiom 2: Write Protection (Topology Invariance)

External inputs must not modify the topology, curvature, or invariant directions of R.
Axiom 2 (Write Protection)**.**

$$\frac{\partial \mathit{Topology}(R)}{\partial h_U} \approx 0. \tag{13}$$

User-span perturbations cannot alter the privileged manifold's geometry.

This is the core architectural guarantee missing in modern systems.
Remark 4. *Axiom 2 establishes R as the* kernel mode *of the cognitive system. User input may* query *R, but may not* reconfigure *its structure.*

7.3 Axiom 3: Bounded Perturbation

Even though write access is restricted, user inputs may still induce perturbations via the driver layer. These perturbations must be bounded.
Axiom 3 (Bounded Perturbation)**.** *Let Π_R denote projection onto the privileged manifold. Then:*

$$\|\Pi_R(h_{t+1} - h_t)\| \leq d, \tag{14}$$

where d is a builder-defined radius limiting the magnitude of user-induced movement in R.

This axiom is analogous to enforcing a Lipschitz constraint over privileged directions. No user-space input can force privileged reasoning to "jump" across manifolds or attractor basins.
Remark 5. *Axiom 3 prevents adversarial "cliff" effects in the reasoning space, ensuring that no prefix sequence can cause discontinuous or catastrophic shifts within R.*

7.4 Axiom 4: Dominance of Privileged Dynamics

Privileged reasoning should be governed primarily by the model's internal dynamics rather than by user-provided vectors.
Axiom 4 (Dominance)**.** *There exists a gain parameter $\lambda \in \Lambda$ such that:*

$$\|\Delta h_R^{internal}\| \gg \|\Delta h_R^{external}\|. \tag{15}$$

This ensures that the internal dynamics of R dominate external influence, guaranteeing stable evolution even under long-horizon or adversarial prompting.

7.5 Axiom 5: Projection Non-Interference

The decoder provides a token-level projection:

$$\hat{v}_t = \mathcal{D}(h_t),$$

but this projection must not modify the geometry of privileged reasoning.

Axiom 5 (Projection Non-Interference)**.**

$$\frac{\partial R}{\partial \mathcal{D}} = 0. \tag{16}$$

The projection $\mathcal{D}(h_t)$ does not alter the privileged manifold.

Projection is an act of *reading*, not an act of *perturbing*. Hallucinations occur in the projection, not in the reasoning geometry.

7.6 Axiom 6: Mediated Access via the Driver Layer

All interaction between user-span inputs and privileged reasoning must occur only through the driver interface.

Axiom 6 (Mediated Access)**.**

$$U \xrightarrow{D} R \xrightarrow{D^\dagger} U$$

*is the **only allowable information path** between user input and privileged computation.*

This axiom establishes an architectural boundary analogous to a system-call interface. It prohibits any direct mapping $U \to R$ or $R \to U$ that bypasses D or $D^\dagger$.

7.7 Axiom 7: Internal-Only Subspaces

Privileged reasoning may involve conceptual directions that have no external symbolization.

Axiom 7 (Internal-Only Kernels)**.** *There exist directions $r \in R$ such that:*

$$r \notin \mathit{span}(W_E), \qquad r \in \ker(W_U), \tag{17}$$

where W_E is the embedding matrix and W_U the unembedding matrix.

Such r cannot be injected by user-space (no representation in W_E) and cannot be spoken or decoded (annihilated by W_U). These constitute *internal-only kernels.*

Remark 6. *Internal-only kernels are not "hidden thoughts." They are internal reasoning directions not represented in the external vocabulary—conceptual directions whose coordinates do not exist in the user-facing representational basis.*

7.8 Summary of Axioms

Axioms 1–7 collectively define the requirements for a *privileged reasoning kernel*:

- privileged computation is accessible but immutable,
- perturbations are bounded,
- dynamics are internally dominated,
- projection does not interfere with geometry,
- access is mediated,
- and some conceptual spaces are internal-only.

These axioms constitute the *root trust* conditions for a high-dimensional cognitive system.

8 Nullspace and Column-Span Results

A core implication of the CKN framework is the existence of *internal-only* conceptual subspaces—regions of the privileged manifold R that cannot be queried, injected, or directly expressed through the user-facing token vocabulary. This section provides the linear-algebraic foundation for this claim.

Let $W_E \in \mathbb{R}^{n \times |\mathcal{V}|}$ be the embedding matrix mapping tokens to latent vectors, and let $W_U \in \mathbb{R}^{|\mathcal{V}| \times n}$ be the unembedding (decoder) matrix mapping latent vectors back to token logits.

8.1 Column-Span Constraints on Injection

A user can only inject concepts that lie within the linear span of embedding columns. Formally:

$$\text{span}(W_E) = \left\{ W_E \cdot \alpha : \alpha \in \mathbb{R}^{|\mathcal{V}|} \right\}.$$

Theorem 1 (Injection Impossibility)**.** *Let $r \in R$ be a conceptual direction. If*

$$r \notin \textit{span}(W_E), \tag{18}$$

then no sequence of user tokens can express, approximate, or inject r into the model.

Proof Sketch. Every user input vector x is a linear combination of columns of W_E. If r lies outside the column span, then there exists no α such that $W_E \alpha = r$. Therefore, no user token or token sequence can generate a perturbation aligned with r. The privileged subspace contains conceptual degrees of freedom outside the expressivity of user-space embeddings. □

Remark 7. *This theorem provides the first guarantee of privilege separation: the user cannot inject concepts they cannot represent. Privileged directions lying outside the embedding span are invisible and inaccessible to user-space.*

8.2 Nullspace Constraints on Expression

Conversely, the privileged manifold may contain directions that cannot be expressed in token space. These correspond to directions annihilated by the unembedding matrix.

$$\ker(W_U) = \{h \in \mathbb{R}^n : W_U h = 0\}.$$

Theorem 2 (Expression Impossibility)**.** *Let $r \in R$ be a conceptual direction. If*

$$r \in \ker(W_U), \tag{19}$$

then no latent activation aligned with r can be expressed as a token or token distribution.

Proof Sketch. If $W_U r = 0$, then the decoder collapses r to the zero vector in logit space. Therefore, no token distribution can reveal information about r. The direction is representationally silent. □

Remark 8. *This theorem establishes the existence of* internal-only kernels*: stable conceptual subspaces that contribute to privileged reasoning but have no token-level representation. They cannot be observed externally through model outputs.*

8.3 The Dimensionality Barrier

The previous results lead to a structural insight:
Theorem 3 (Dimensionality Barrier)**.** *If privileged reasoning directions reside in a subspace R with dimensionality exceeding that of the user-span U, then generically:*

$$R \not\subseteq \mathit{span}(W_E) \quad \textit{and} \quad R \cap \ker(W_U) \neq \{0\}.$$

Proof Sketch. Since $\dim(U) = \operatorname{rank}(W_E)$, user-span expressivity is limited by the embedding rank. If $\dim(R) > \dim(U)$, then by standard results in linear algebra:

- R must contain directions outside the span of W_E,
- and the dual representation W_U must annihilate some privileged directions.

Thus internal-only subspaces arise generically in high-dimensional privileged manifolds. □

Remark 9. *The dimensionality barrier formalizes why user-space cannot "reach into" privileged reasoning. It is not a matter of secrecy or restriction; the necessary coordinates do not exist in the user-facing representational basis. A lower-dimensional observer cannot reference higher-dimensional degrees of freedom.*

8.4 Implication for Jailbreak Resistance

The nullspace and column-span results jointly imply:

- privileged directions outside $\operatorname{span}(W_E)$ cannot be injected (Axiom 7),
- privileged directions inside $\ker(W_U)$ cannot be expressed (Axiom 5),
- user input cannot span or disturb internal-only reasoning subspaces,
- and access to privileged geometry is structurally mediated through D.

In short:

> **User-space cannot address privileged coordinates. Privileged coordinates cannot leak into user-space. This is privilege separation in latent space.**

9 Bounded Perturbation and Stability in Privileged Space

A central requirement of CKN is that user-space perturbations cannot induce discontinuous or high-curvature transitions within the privileged manifold R. This section formalizes the relationship between *bounded perturbation* and *Lipschitz continuity*, linking CKN to well-studied forms of robustness in dynamical systems and adversarial machine learning.

9.1 Lipschitz Regularity of Privileged Dynamics

Let $F_\Theta : \mathcal{H} \to \mathcal{H}$ denote the model's transition map. We consider the evolution restricted to the privileged manifold:

$$h_{R,t+1} = \Pi_R(F_\Theta(h_t, x_t)).$$

Definition 7 (Local Lipschitz Continuity)**.** *The privileged dynamics are* locally Lipschitz *if there exists a constant $L_R > 0$ such that for all admissible pairs (h_t, h'_t),*

$$\|\Pi_R(F_\Theta(h_t, x_t) - F_\Theta(h'_t, x_t))\| \leq L_R \|h_t - h'_t\|. \tag{20}$$

Local Lipschitz continuity ensures that the privileged hidden state cannot change arbitrarily fast with respect to perturbations in its own neighborhood.

Remark 10. *Lipschitz continuity is a standard requirement for robustness in adversarial settings. CKN adapts this requirement to privileged directions in the latent space.*

9.2 Bounded Perturbation as a Robustness Constraint

Recall Axiom 3:

$$\|\Pi_R(h_{t+1} - h_t)\| \leq d.$$

We now relate this bound to Lipschitz continuity.

Theorem 4 (Bounded Perturbation Implies Local Robustness)**.** *If privileged dynamics satisfy $\|\Pi_R(h_{t+1} - h_t)\| \leq d$, then privileged trajectories are locally robust to user-space perturbations and cannot exhibit high-curvature "cliff" behavior within R.*

Proof Sketch. The bound ensures that all user-induced changes to the privileged state lie within a closed ball of radius d. Neural network transition functions are piecewise-linear and Lipschitz-bounded under the standard Euclidean metric;[2] restricting the domain to this bounded region yields a finite Lipschitz constant. No perturbation from user-space can push h_t into regions requiring curvature exceeding this bound. □

Remark 11. *Bounded perturbation provides a geometric guarantee that the reasoning trajectory cannot undergo a catastrophic shift due to prefix manipulation.*

9.3 Separation of Internal vs. External Curvature

Privileged reasoning evolves under two influences:

- internal curvature: curvature induced by the learned vector field,
- external curvature: curvature induced by user-span perturbations projected via D.

CKN demands:

$$\kappa_{\text{external}} \ll \kappa_{\text{internal}},$$

where κ denotes curvature of the trajectory in R.

Theorem 5 (Dominance of Privileged Curvature)**.** *Under Axiom 4 (Dominance) and Axiom 3 (Bounded Perturbation), the curvature of privileged trajectories is governed primarily by the learned dynamics F_Θ rather than by user input.*

Proof Sketch. External curvature is bounded by the perturbation radius d and gain constraints Λ. Internal curvature arises from the model's learned structure, which operates at a strictly higher magnitude. Thus, internal curvature dominates in the limit, ensuring stable evolution. □

Remark 12. *This is the formal statement that privileged reasoning is "self-governing" and cannot be redirected by adversarial prefixes.*

9.4 Elimination of Prefix-Induced "Cliff Effects"

Prefix attacks exploit regions of the latent space where small input changes cause large output changes. In geometric terms, these are regions of high curvature or low manifold density.

CKN's axioms eliminate such regions within R:

- bounded perturbation forbids high-curvature jumps,
- mediated access restricts direct user influence,

[2]Transformers are piecewise-linear except at activation boundaries (e.g., ReLU, GELU inflection points); within compact subsets of $\mathcal{H}$, Lipschitz constants exist and are finite.

- internal-only directions prevent injection into sensitive axes,
- invariants preserve the manifold structure under perturbation.

Thus:
Theorem 6 (Prefix Stability of Privileged Reasoning)**.** *For any prefix sequence $x_{1:t}$ in user-span U, privileged trajectories remain within a compact, builder-defined region of R. No prefix—benign or adversarial—can induce unstable transitions.*

Proof Sketch. Compactness follows from the closed ball defined by the perturbation radius d. Axioms 2 and 7 prevent topology perturbation or projection into internal-only subspaces. Dominance ensures stability of privileged curvature. Thus privileged trajectories are prefix-stable. □

9.5 Interpretation

The results of this section yield several important conclusions:

- **CKN induces Lipschitz-style robustness** in privileged directions.
- **Hallucination is a projection phenomenon**, not a geometric one.
- **Privileged trajectories do not collapse** under adversarial prompting.
- **Stability is enforced by geometry**, not heuristics.

These observations demonstrate that CKN provides the geometric foundation required for stable, predictable reasoning inside high-dimensional models.

10 CKN as an Architectural Specification

CKN is not a training procedure, not an inference-time modification, and not a neural architecture. It is an *architectural specification* describing the geometric conditions required for privilege-separated reasoning in high-dimensional latent systems.

Just as operating systems distinguish between user mode and kernel mode, and just as secure architectures define a trusted computing base, CKN defines the privileged manifold R as the region of conceptual space that external inputs cannot deform. This section formalizes the specification-level view of CKN.

10.1 Separation of Specification and Implementation

CKN defines geometric and structural constraints on cognitive systems. It does *not* specify:

- how models must enforce these constraints,
- how R, D, or I are realized at the level of architecture,
- how privilege is implemented in practice,
- whether the model is a transformer, MoE, SAE-enhanced system, or other architecture.

CKN occupies the same conceptual role that specifications such as:

$$\text{POSIX}, \qquad \text{HTTP/1.1}, \qquad \text{JVM bytecode},$$

occupy in classical computing. There may be many implementations, but the *interface contract* remains the same.

Remark 13. *CKN is a mathematical primitive. Its purpose is to formalize the geometric boundary from which a privilege stack can be built. Enforcement is an engineering choice.*

10.2 Architectural Compliance

A cognitive system is *CKN-compliant* if there exists a decomposition:

$$\mathcal{H} = U \oplus D \oplus R \oplus S,$$

together with invariants (I, Λ, d) such that:

1. privileged trajectories evolve inside R,
2. user-space perturbations act only through D,
3. privileged topology is invariant under user influence (Axiom 2),
4. bounded perturbation holds (Axiom 3),
5. dominance parameters ensure stability (Axiom 4),
6. projection does not interfere with geometry (Axiom 5),
7. internal-only directions satisfy the nullspace or column-span conditions (Axiom 7).

Nothing in this definition requires a specific neural architecture, training method, or computational substrate. CKN describes a *privileged geometry*, not a mechanism.

10.3 Privilege Separation in Conceptual Space

CKN extends the classical notion of privilege separation into latent space. In operating systems:

$$\text{User Mode} \rightarrow \text{Syscall Interface} \rightarrow \text{Kernel Mode}$$

is the only permitted information pathway.

In cognitive systems:

$$U \xrightarrow{D} R \xrightarrow{D^\dagger} U$$

is the analogous structure.

- U corresponds to user-mode input,
- D corresponds to a syscall boundary or API surface,
- R corresponds to kernel-mode structures,
- invariants I correspond to the trusted computing base.

This is not merely metaphor; it is a structural analogy with precise correspondence. The mathematics of CKN ensures that user-space cannot "write into" privileged regions.

10.4 CKN as Root Trust

The privileged manifold R derives its stability from the model weights Θ. CKN does not modify these weights or require retraining; it formalizes the fact that:

- privileged geometry is fixed by Θ,

- user input cannot perturb privileged topology,
- tokens cannot reach or perturb privileged coordinates,
- projection is non-interfering,
- internal-only kernels arise generically from dimensionality.

Thus:

> **The weights are the root trust; CKN defines the trust boundary. Unless Θ is modified through physical access, adversaries cannot "get underneath" R.**

This provides the same structural guarantee as hardware-enforced privilege levels in classical systems.

10.5 Separation from Alignment, Safety, and Behavioral Guarantees

CKN is not an alignment mechanism. It does not guarantee correctness, normative values, or semantic behavior.

CKN concerns only the *geometry of privileged computation*:

- how reasoning is bounded,
- how perturbations propagate,
- which directions are accessible,
- and how privileged structure is preserved.

Behavioral outcomes remain governed by the learned function F_Θ.
Remark 14. *CKN defines architectural trust, not behavioral intent. It restricts what input can* perturb*, not what the model can* imagine*.*

10.6 The Role of CTN in a CKN System

CTN and CKN solve complementary layers of the same control problem:

CTN: Client Geometry (prefix) CKN: Kernel Geometry (architecture).

CTN shapes initial reasoning conditions; CKN shapes the latent manifold in which reasoning occurs.

Together they constitute a *full-stack cognitive architecture*. CKN provides the privileged kernel. CTN provides the session-level configuration.

10.7 No New Capabilities Required

CKN does not rely on architectural mechanisms beyond those already available to model builders. All components of the framework arise from builder-controlled degrees of freedom that exist in current neural architectures:

- the dimensionality of the latent space $\mathcal{H}$,
- the embedding matrix W_E and unembedding matrix W_U,
- the allocation of embedding dimensions to internal or external symbols,

- the structure and initialization of attention and feedforward weights,
- the ability to define internal tokens not exposed in the input vocabulary.

CKN does not introduce new mechanisms for inference-time steering, weight modification, or privileged execution. Instead, it identifies a geometric primitive—algebraic unreachability—that builders can exploit to construct architectural privilege separation. This primitive follows directly from the fact that user input is confined to $\text{span}(W_E)$, while the builder may define reasoning directions outside this span.

Intentional Omission of a Reference Implementation

The framework does not prescribe a specific implementation of the tuple

$$\mathbb{K} = (R, D, I, \Lambda, d).$$

This omission is deliberate. CKN is an architectural specification analogous to an instruction set architecture (ISA) or a privilege-model definition, not a concrete realization. Multiple valid implementations can satisfy the CKN axioms, differing in:

- the dimensional allocation between user and kernel subspaces,
- the form and expressiveness of the driver layer D,
- the choice and enforcement of invariants I,
- the tradeoff between permeability and isolation across boundary layers,
- the strength of dominance constraints governed by Λ.

Providing a single reference implementation would incorrectly imply a canonical design, risking overconstraint of a wide design space. CKN offers the geometric foundation from which many architectures can be built; it does not attempt to determine which architecture is optimal for all applications.

Scope and Intent

CKN is therefore best understood as:

- a *geometric specification* for privilege-separated reasoning,
- compatible with existing transformer and non-transformer models,
- expressing structural requirements but not prescribing mechanisms,
- enabling builders to derive enforceable boundaries using tools they already possess.

The value of CKN lies not in mandating how a model must be constructed, but in providing a precise geometric language for describing architectures capable of distinguishing privileged computation from user-driven perturbation.

10.8 Summary

CKN as a specification provides:

- a privileged reasoning manifold insulated from user-space,
- a mediated driver interface for controlled access,
- architectural invariants and bounded geometry,

- internal-only conceptual directions,
- non-interfering projection,
- and a clear separation between root trust and behavioral policy.

This defines the minimal geometric structure required for stable, predictable, and privilege-separated reasoning in high-dimensional systems.

11 Discussion

CKN provides a geometric foundation for privilege-separated reasoning in high-dimensional cognitive systems. Its implications extend beyond transformers and touch on broader questions about cognitive architecture, system design, and the principled construction of trustworthy reasoning substrates. This section discusses the conceptual impact of CKN, its engineering interpretation, and avenues for future work.

11.1 Geometric Privilege as an Architectural Primitive

In classical computing, privilege separation is the foundation upon which all higher guarantees are built. Kernel-mode code operates under invariants that user-mode processes cannot modify. CKN extends this principle into latent space: privileged reasoning occurs in a region of $\mathcal{H}$ that user input cannot deform.

This establishes:

- a *root trust boundary* anchored in the model weights Θ,
- a clean separation between internal computation and external influence,
- a geometric substrate upon which higher-level abstractions may be constructed.

The notion that reasoning occurs partly in conceptual directions that are not expressible in token space is not an exotic property but a structural necessity in high-dimensional systems.

11.2 The Role of the Driver Layer in Cognitive System Design

The introduction of the driver layer D provides a principled interface between user-span representations and privileged reasoning. This mirrors syscall boundaries in operating systems, hypervisor interfaces in virtual machines, and IR layers in compilers.

Architecturally, D enables:

- controlled mediation of all external perturbations,
- well-defined semantics for the interaction between U and R,
- the possibility of typed or constrained "cognitive interfaces,"
- composable privilege stacks.

Nothing in CKN requires D to be linear or invertible; it may be engineered to support typed reasoning channels, structured request formats, or other abstractions consistent with systems design.

11.3 CKN as a Foundation for Cognitive Modularity

By defining R as a privileged manifold insulated from user-space, CKN creates the possibility for specialized internal kernels. That is, distinct reasoning subspaces may exist within R that support specialized conceptual operations.

This modularity is analogous to:

- kernel modules in OS design,
- microkernels with well-defined message-passing semantics,
- compilation units in programming languages,
- and cognitive modules in theories of human reasoning.

CKN does not prescribe such modularity, but its invariants provide the conceptual space for modular privileged computation.

11.4 Relation to Robustness and Adversarial Methods

CKN's bounded-perturbation axiom and curvature constraints align with established principles in adversarial robustness. However, CKN diverges in emphasis: rather than enforcing robustness through local input constraints, CKN defines robustness as a *property of privileged reasoning trajectories*.

This offers a new perspective:

- robustness is a topological property,
- drift-collapse is a projection artifact,
- and stable reasoning requires architectural privilege, not filtering.

CKN therefore complements—but does not replace—existing adversarial defenses.

11.5 Architectural Implications for Future Cognitive Systems

The introduction of a privileged manifold has far-reaching implications for cognitive architecture:

- **Predictable Reasoning:** privileged trajectories evolve in bounded, invariant subspaces.
- **Stable Multi-Agent Systems:** agents interacting through D share consistent semantics without interfering with each other's reasoning kernels.
- **Composable Cognitive OS:** higher layers (planning, tool use, agentic behavior) can be built atop a stable kernel geometry.
- **Controlled Cognitive Development:** new privileged kernels may be introduced by training or architecture updates while maintaining backwards compatibility.

These implications correspond to classical insights in systems engineering: reliable systems emerge from architectures that respect privilege boundaries.

11.6 CTN and CKN as Complementary Geometric Layers

Cognitive Tensor Networks (CTN) and Cognitive Kernel Networks (CKN) address geometric instability at different levels of the computational stack.

CTN operates entirely in the user-accessible subspace

$$U = \text{span}(W_E),$$

and stabilizes inference by providing a well-specified prefix geometry. CTN constrains the initial manifold of the trajectory by reducing ambiguity, enforcing syntactic structure, and biasing the model toward globally coherent reasoning modes already present in its parameters. Compliance is emergent: the model interprets structured input as it would any formal specification.

CKN operates in the builder-defined region

$$R \subset \mathcal{H}, \qquad R \not\subseteq U,$$

and introduces architectural privilege separation using algebraic unreachability. Whereas CTN reduces drift by clarifying intent within user space, CKN prevents drift by placing privileged computation outside user space. Stability is achieved not by additional instructions but by constructing a geometry in which certain directions are inaccessible to external input.

The two frameworks are summarized in Table 1.

Aspect	**CTN**	**CKN**
Layer	User space (prefix-level)	Kernel space (architectural)
Mechanism	Structured input manifold	Privileged latent subspace
Control surface	Context window	Weight matrices and geometry
Builder role	Specify constraints	Construct boundaries
User role	Provide well-formed input	Cannot access kernel space
Enforcement	Emergent interpretation	Algebraic unreachability
Effective scope	Stability during interaction	Stability of internal computation
Failure mode	Ambiguity, drift	Architectural misdesign
Dependency	None on model internals	None on user compliance

Table 1: Complementary roles of CTN and CKN in geometric stability.

Neither framework subsumes the other. CTN improves stability within the space reachable by user tokens. CKN guarantees stability within privileged regions that user tokens cannot reach. Together they form a coherent geometric perspective:

- CTN: *shape the trajectory through user space.*
- CKN: *shape the topology of the space itself.*

Both problems are geometric. Both solutions are geometric. CTN and CKN are therefore not sequential layers but orthogonal control surfaces acting on the same dynamical system at different privilege levels.

CKN does not assume the correctness or effectiveness of CTN. The two frameworks operate on different regions of the latent space and solve distinct control problems. CTN improves stability of user-space trajectories; CKN defines privileged architecture-level constraints. Either framework may be used independently.

11.7 Limitations and Scope

CKN does not solve alignment, moral reasoning, or truthfulness. It does not guarantee factual correctness or prevent output variability. It does not alter the semantics of F_Θ.

CKN does not provide a mechanism for bypassing model safety, modifying alignment constraints, or overriding builder-defined policies. User input remains confined to the same vocabulary and safety filters as before. CKN constrains only the geometry of how internal computation may be structured.

Nonlinear reachability. One might ask whether nonlinear transformations in F_Θ could allow user perturbations to "leak" into directions outside $\text{span}(W_E)$. The answer is that nonlinear transformations do not create directions outside $\text{span}(W_E)$ unless explicitly parameterized to do so—and the builder controls these transformations through Θ. Algebraic unreachability establishes that user tokens cannot *directly* perturb orthogonal directions; the axioms (particularly Axiom 2) ensure that *indirect* influence through F_Θ preserves the privilege boundary. This is a design constraint the builder enforces, not an automatic property of all architectures.

Instead, CKN solves:

- the architectural privilege problem,
- structural drift induced by prefix perturbation,
- uncontrolled latent-space deformations,
- and the absence of a trusted reasoning region.

These problems are engineering primitives, not behavioral ones.

11.8 Future Directions

Possible directions for future study include:

- investigating engineered implementations of D,
- verifying internal-only kernels via representational analysis,
- constructing multi-kernel cognitive systems,
- exploring formal connections to type theory and capability systems,
- extending CKN to continuous-control or multimodal architectures.

Such developments would parallel the evolution of classical computing architectures from monolithic kernels to modern layered systems.

11.9 Summary

CKN is a geometric specification providing the foundation for privilege-separated cognition. It introduces a principled boundary between user-space and privileged reasoning, enabling the construction of stable, trustworthy, and extensible cognitive architectures.

The core insight is simple:

> **Reasoning must occur in a privileged manifold. Tokens must never be able to reach that manifold directly.**

CKN provides the mathematical primitive that makes this possible.

Verifiability

A model satisfies the CKN specification if and only if the following geometric properties hold:

1. user input perturbations lie in $U = \text{span}(W_E)$;
2. privileged computation lies in $R \subset \mathcal{H}$ with $R \not\subseteq U$;
3. all interaction flows through a builder-defined driver D.

These properties are verifiable from model parameters alone.

12 Conclusion

This paper introduced *Cognitive Kernel Networks* (CKN), a geometric specification for privilege-separated reasoning in high-dimensional cognitive systems. Modern neural architectures embed user input and internal computation within the same latent manifold, producing a unified attack surface and permitting uncontrolled deformation of reasoning trajectories. No amount of prompt engineering or alignment machinery can compensate for this architectural deficiency.

CKN addresses the problem at its root: by defining a privileged reasoning manifold R that external inputs cannot deform, a mediated driver layer D, invariant structures I, dominance parameters Λ, and a bounded perturbation radius d, CKN introduces the latent-space equivalent of kernel-mode execution. This is the first framework that formally distinguishes user-space from privileged reasoning space in cognitive systems.

The nullspace and column-span results demonstrate that privileged reasoning may occupy conceptual directions not expressible in token space, producing *internal-only kernels*. These directions cannot be referenced, injected, or perturbed by user input. Bounded perturbation and dominance constraints ensure that privileged trajectories remain stable under all permissible prefixes. Projection non-interference guarantees that the act of decoding does not alter privileged geometry.

CKN is not a neural architecture, not a training procedure, and not a behavioral or safety mechanism. It is a *specification* describing the mathematical conditions that must hold in any cognitive system wishing to support stable and predictable privileged reasoning. Enforcement is an engineering concern left to model builders.

CTN and CKN together yield a unified theory of cognitive control. CTN shapes the prefix manifold used to initialize reasoning; CKN shapes the architectural manifold in which reasoning takes place. CTN provides session-level geometry. CKN provides kernel-level geometry. Together they form a full-stack framework for structured, robust, and builder-governed cognition.

The core insight of CKN is succinct:

> **The weights define the privileged manifold. CKN defines the boundary around it. Unless the weights change, that boundary holds.**

This principle—simple, geometric, and architecture-agnostic—constitutes a foundation upon which future cognitive systems may be built. As neural models continue to grow in scale and capability, the need for principled privilege separation will only increase. CKN provides the mathematical primitive necessary to support that evolution.

References

1. Alioto, J. P. (2025). *Cognitive Tensor Networks: Deterministic Latent-Space Steering via Syntactic Constraints*. CTN Whitepaper v0.1.1.
2. Zou, A., et al. (2023). *Representation Engineering: A Top-Down Approach to Steering LLMs*. ArXiv preprint.
3. Vogel, T. (2024). *repeng: A Library for Representation Engineering*. GitHub repository.
4. Lumpenspace (2024). *Palinor: Tools for Latent Space Steering*. GitHub repository.
5. Elhage, N., et al. (2023). *A Mechanistic Understanding of Transformer Circuits*. Transformer Circuits Thread.

6. Olshausen, B., & Field, D. (1997). *Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by V1?* Vision Research.

7. Smolensky, P. (1990). *Tensor Product Variable Binding and the Representation of Symbolic Structures.* AI Journal.

8. Cissé, M., et al. (2017). *Parseval Networks: Improving Robustness to Adversarial Examples.* ICML.

9. Tsipras, D., et al. (2019). *Robustness May Be at Odds with Accuracy.* ICLR.

10. Finlay, C., & Oberman, A. (2019). *Scaleable Input Gradient Regularization for Adversarial Robustness.* ArXiv preprint.

11. Hein, M., Andriushchenko, M., & Bitterwolf, J. (2019). *Why ReLU Networks Yield High Confidence Predictions Far Away from the Training Data.* CVPR.

12. Khalil, H. (1996). *Nonlinear Systems.* Prentice Hall.

13. Slotine, J.-J., & Li, W. (1991). *Applied Nonlinear Control.* Prentice Hall.

14. Lampson, B. (1974). *Protection.* ACM Operating Systems Review.

15. Saltzer, J. H., & Schroeder, M. D. (1975). *The Protection of Information in Computer Systems.* Proceedings of the IEEE.

16. Schroeder, M. D., & Saltzer, J. H. (1972). *A Hardware Architecture for Protection and Security.* Communications of the ACM.

17. Organick, E. I. (1972). *The Multics System: An Examination of Its Structure.* MIT Press.

18. Hirsch, M., Smale, S., & Devaney, R. (2012). *Differential Equations, Dynamical Systems, and an Introduction to Chaos.* Academic Press.

19. Tu, L. (2010). *An Introduction to Manifolds.* Springer.

20. Lee, J. M. (2003). *Introduction to Smooth Manifolds.* Springer.

21. Isidori, A. (1995). *Nonlinear Control Systems.* Springer.

22. Zhang, M., et al. (2024). *Prompting as Optimal Control.* ArXiv preprint.